

Relatório - Classificação de Tumores Cerebrais em Imagens de RM

Aluno: Rodolfo Oliveira Miranda

1. Introdução

O objetivo deste trabalho é comparar duas técnicas de Aprendizado de Máquina - CNN (Convolutional Neural Network) e Vision Transformer (ViT) - na tarefa de classificar imagens de RM cerebral em 4 classes. Foram realizados 8 experimentos no total (4 de por técnica), variando técnicas de redução de overfitting (Dropout, regularização L2/weight decay, data augmentation) e hiperparâmetros (número de filtros/neurônios, número de heads/layers do Transformer), avaliando o impacto de cada escolha sobre acurácia, precisão, recall e F-Score.

2. Dataset

Fonte: Kaggle - '[masoudnickparvar/brain-tumor-mri-dataset](https://www.kaggle.com/masoudnickparvar/brain-tumor-mri-dataset)'

Classes: glioma, meningioma, tumor hipofisário (pituitary), sem tumor (notumor).

Treino: 5.600 imagens (1.400 por classe), das quais 85% usadas para treino efetivo e 15% separadas para validação durante o treinamento.

Teste: 1.600 imagens (400 por classe), mantidas completamente isoladas e usadas apenas na avaliação final de cada experimento (held-out).

Pré-processamento: conversão para escala de cinza (MRI não tem informação de cor a perder), redimensionamento para **128x128 pixels** e normalização dos valores de pixel para o intervalo [0,1].

3. Metodologia

3.1. Ambiente e orçamento de treino

Foi criado um ambiente dedicado com Python 3.12 e instalado TensorFlow/Keras "clássico" (`tensorflow==2.21.0`, que já inclui Keras 3). Como o TensorFlow nativo do Windows perdeu suporte e aceleração por GPU a partir da versão 2.11, o treinamento dos 8 experimentos foi realizado em CPU.

A resolução das imagens e a profundidade das arquiteturas (4 blocos convolucionais com Batch Normalization na CNN; 8 camadas de atenção no ViT) foram escolhidas pelo que é adequado à tarefa. Dado o custo computacional de treinar 8 modelos em CPU, mediu-se empiricamente - via testes de fumaça - o tempo por época de cada arquitetura antes de definir o orçamento de treinamento, chegando a um teto de 10 épocas com `EarlyStopping` de paciência 4 (restaura os pesos de melhor época por `val_loss`). Na prática, todos os 8 experimentos usaram as 10 épocas

completas - nenhum disparou a parada antecipada - ou seja, o teto de épocas foi o fato limitante real, não a convergência do modelo.

3.2. Técnicas de ML utilizadas

CNN: 4 blocos `Conv2D` (32→64→128→256 filtros) + `BatchNormalization` + `MaxPooling2D`, seguidos de `GlobalAveragePooling2D`, uma camada densa e saída `softmax` de 4 classes.

Implementação em `[src/models/cnn.py](../src/models/cnn.py)`.

Vit: a imagem é dividida em patches de 16×16 (extraídos via `Conv2D` com stride igual ao tamanho do patch — 64 patches por imagem), projetados em um embedding de 128 dimensões com posição somada, processados por 8 blocos de Transformer (`MultiHeadAttention` de 8 heads + MLP com conexões residuais e `LayerNormalization`) e classificados após um `GlobalAveragePooling1D`.

Implementação em `[src/models/vit.py](../src/models/vit.py)`.

3.3. Métricas

O enunciado define as métricas para o caso binário. Como o problema é multiclasse (4 classes), cada métrica foi generalizada pela **média macro** entre as 4 classes (cada classe tratada como "positiva" vs. as demais, depois calculada a média não-ponderada):

Acurácia: proporção de previsões corretas sobre o total.

Precisão (macro): média da precisão de cada classe.

Recall (macro): média do recall de cada classe.

F1-Score (macro): média harmônica entre precisão e recall de cada classe.

Cálculo via `sklearn.metrics` (`accuracy_score`, `precision_score`, `recall_score`, `f1_score` com `average="macro"`) sobre conjunto de teste held-out, em `[src/metrics_utils.py](../src/metrics_utils.py)`.

3.4. Os 8 experimentos

ID	Técnica	Configuração	O que está sendo testado
C1	CNN	filtros=(32,64,128,256), sem regularização	baseline
C2	CNN	+ Dropout (0.5)	redução de overfit - dropout
C3	CNN	+ L2 (1e-4) + data augmentation	redução de overfit - L2 + augmentation

C4	CNN	dropout+L2+augmentation, filtros=(48,96,192,384), 384 neurônios	tuning combinado (mais capacidade)
V1	ViT	patch=16, embed=128, heads=8, layers=8, sem dropout	baseline
V2	ViT	+ Dropout (0.3) em atenção e MLP	redução de overfit - dropout
V3	ViT	+ weight decay (AdamW, 1e-4) + data augmentation	redução de overfit - weight decay + augmentation
V4	ViT	dropout+weight decay, embed=160, heads=10, layers=10	tuning combinado (mais capacidade)

4. Trecho de código

4.1. Modelo CN (`build_cnn`)

Python

```
def build_cnn(input_shape=(128, 128, 1), num_classes=4,
conv_filters=(32, 64, 128, 256),
              dense_units=256, dropout_rate=0.0, l2_reg=0.0):
    reg = regularizers.l2(l2_reg) if l2_reg > 0 else None
    inputs = layers.Input(shape=input_shape)
    x = inputs
    for filters in conv_filters:
        x = layers.Conv2D(filters, 3, padding="same",
activation="relu", kernel_regularizer=reg)(x)
        x = layers.BatchNormalization()(x)
        x = layers.MaxPooling2D()(x)
        if dropout_rate > 0:
            x = layers.Dropout(dropout_rate / 2)(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(dense_units, activation="relu",
kernel_regularizer=reg)(x)
    if dropout_rate > 0:
        x = layers.Dropout(dropout_rate)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return tf.keras.Model(inputs, outputs, name="cnn")
```

4.2. Modelo Vit (`build_vit`)

Python

```
def build_vit(input_shape=(128, 128, 1), num_classes=4,
              patch_size=16, embed_dim=128,
              num_heads=8, num_layers=8, mlp_dim=256,
              dropout_rate=0.0):
    num_patches = (input_shape[0] // patch_size) *
        (input_shape[1] // patch_size)
    inputs = layers.Input(shape=input_shape)
    x = PatchEncoder(patch_size, embed_dim, num_patches)(inputs)
    for _ in range(num_layers):
        x = transformer_block(x, embed_dim, num_heads, mlp_dim,
                              dropout_rate)
    x = layers.LayerNormalization(epsilon=1e-6)(x)
    x = layers.GlobalAveragePooling1D()(x)
    if dropout_rate > 0:
        x = layers.Dropout(dropout_rate)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return tf.keras.Model(inputs, outputs, name="vit")
```

Código completo dos experimentos, pipeline de dados e métricas em `[src/](../src/)`.

5. Resultados

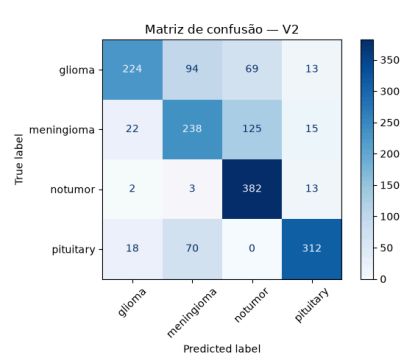
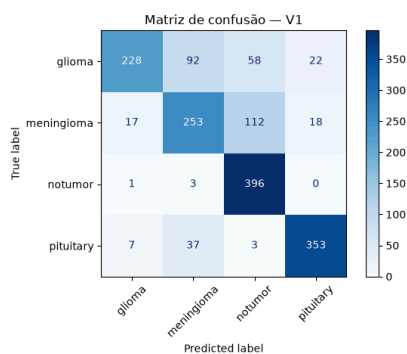
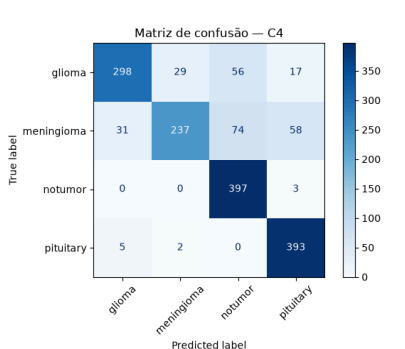
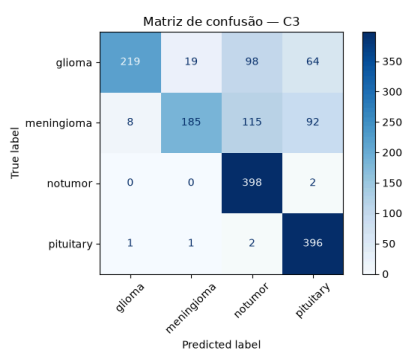
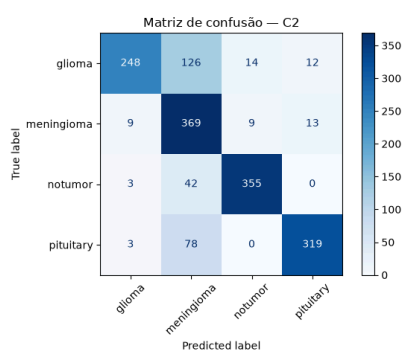
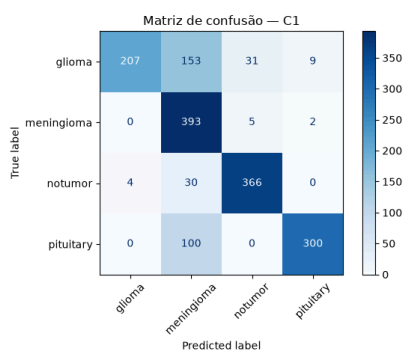
Métricas calculadas sobre o conjunto de teste held-out (1.600 imagens, nunca usadas em treino ou validação). Dados completos em `[results/results.csv](../results/results.csv)`; log integral do treinamento em `[results/training_log.txt](../results/training_log.txt)`.

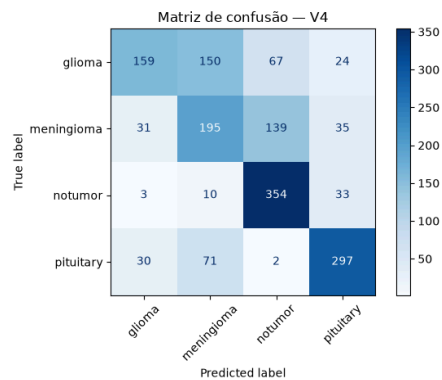
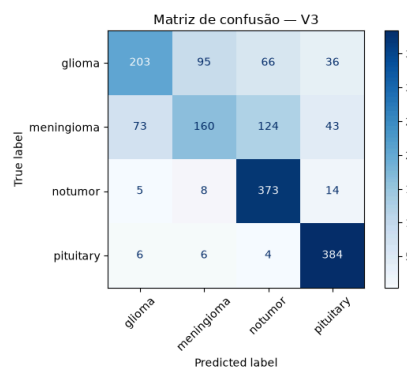
ID	Técnica	Tempo de treino (s)	Acurácia	Precisão (macro)	Recall (macro)	F1 (macro)
C1	CNN baseline	747.6	0.7913	0.8594	0.7913	0.7912
C2	CNN + Dropout	633.3	0.8069	0.8524	0.8065	0.8113
C3	CNN + L2 +augmentation	665.0	0.7488	0.8068	0.7488	0.7312
C4	CNN + tuned	1225.3	0.8281	0.8411	0.8281	0.8221
V1	ViT baseline	375.2	0.7688	0.7881	0.7688	0.7626
V2	ViT + Dropout	391.5	0.7225	0.7442	0.7225	0.7189

V3	ViT + weight decay + augmentation	312.1	0.7000	0.6912	0.7000	0.6791
V4	ViT tuned	596.6	0.6281	0.6410	0.6281	0.6179

Todos os experimentos usaram as 10 épocas completas (nenhum disparou **EarlyStopping** antecipadamente) - ver seção 6 para a discussão sobre essa limitação.

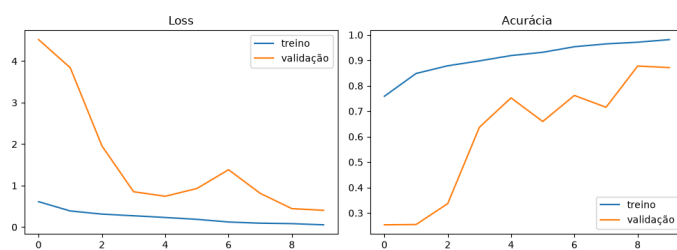
Matrizes de confusão:



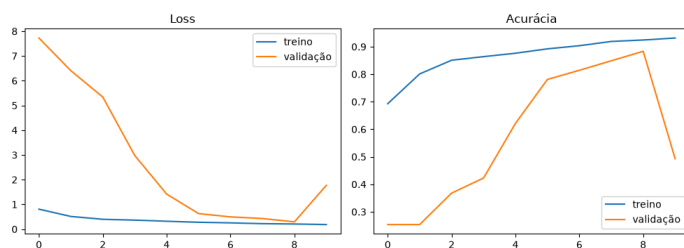


Curvas de treino (loss/acurácia por época):

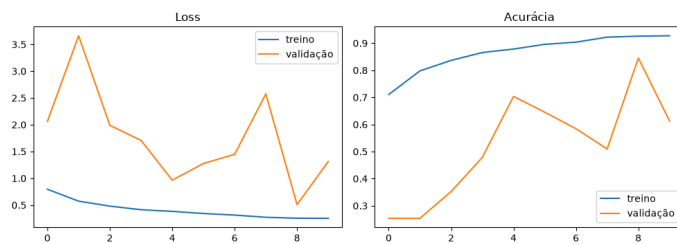
Curvas de treino — C1



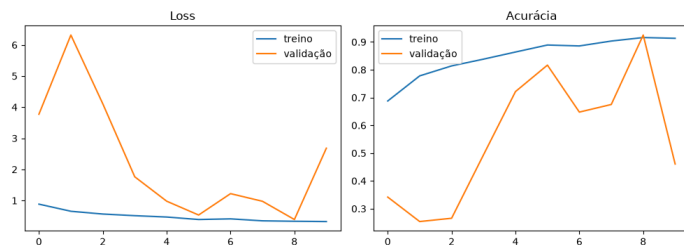
Curvas de treino — C2

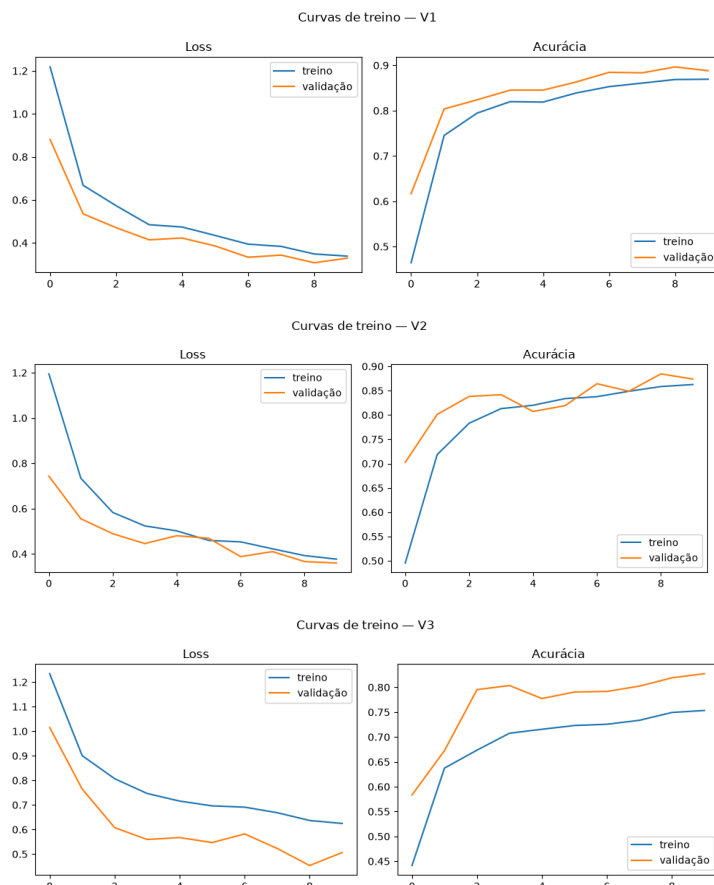


Curvas de treino — C3



Curvas de treino — C4





6. Qual foi a melhor abordagem?

A melhor abordagem entre as 8 testas foi **C4 tuned (Dropout + L2 + Data Augmentation + mais filtros/neurônios)**, com **82,81% de acurácia e F1-macro de 0,820** no conjunto de teste - a única configuração que combina todas as técnicas de regularização e ainda assim melhorou sobre o baseline.

Conjecturas sobre os resultados:

1. **CNN superou o ViT em todos os 4 pares comparáveis**, por margens de 2 a 20 pontos percentuais de acurácia. Isso é esperado: convoluções têm um “viés indutivo” de localidade e invariância à translação embutido na arquitetura, enquanto Transformers precisam aprender essas regularidade a partir de dados (ou de pré-treinamento em larga escala). Com ~4.760 imagens de treino e apenas 10 épocas, o ViT não teve dados nem tempo suficiente para compensar essa desvantagem estrutural - mesmo sendo computacionalmente mais barato por época que a CNN nesta resolução (45-99s/época vs. 67-120s/época), o ViT não converteu esse menor custo em melhor acurácia.
2. **Mais capacidade ajudou a CNN, desta vez.** C4 (o experimento com mais filtros/neurônios, combinado com dropout+L2+augmentation) foi o melhor de todos os 8 - diferente de uma rodada anterior deste mesmo trabalho (com imagens menores e paciência de parada antecipada mais curta), em que o equivalente “tuned” havia sido o **pior** resultado. A diferença está no orçamento

de épocas: com paciência maior e o teto de 10 épocas sendo usado por completo nesse experimento, o modelo maior teve mais chances de aproveitar sua capacidade extra antes da parada - capacidade adicional só compensa quando há orçamento de treino suficiente para convergir.

3. **C3 (L2 + augmentation, sem dropout) foi o pior resultado entre as CNNs.** Uma conjectura que a combinação de data augmentation com um orçamento de apenas 10 épocas tornou a tarefa de treino mais difícil sem dar tempo suficiente para o modelo se beneficiar da regularização: a augmentation aumenta a variabilidade efetiva dos dados de treino, o que tende a ajudar a generalização em treinos longos, mas pode atrasar a convergência em treinos curtos.
4. **No ViT, toda regularização adicional piorou o resultado** ($V1 > V2 > V3 > V4$, monotonicamente). Isso sugere que o ViT, dentro de apenas 10 épocas, ainda estava na fase inicial de aprendizado (“warm-up”) - fenômeno bem documentado para Transformers, que tipicamente precisam de mais épocas/dados para começar a generalizar bem. Adicionar dropout, weight decay ou mais parâmetros ($V4$) só tornou essa fase inicial mais lenta, sem o modelo ter tempo “compensar” depois.
5. **Instabilidade nas primeiras épocas.** As curvas de treino (`results/training_curvers/`) mostram a acurácia de validação das CNNs caindo a ~25% (nível de chance entre 4 classes) nas primeiras 1-3 épocas, mesmo enquanto a acurácia de treino já subia - um efeito conhecido da interação entre `BatchNormalization` (cuas estatísticas de média/variância levar algumas épocas para estabilizar) e um orçamento de treino curto. O uso de `EarlyStopping(restore_best_weights=True)` monitoramento `val_loss` evita que esse ruído inicial prejudique a avaliação final, pois os pesos usados na avaliação são os da melhor época, não necessariamente os da última.

Limitação principal: nenhum dos 8 experimentos convergiu por completo — todos usaram o teto de 10 épocas sem disparar parada antecipada. Esse teto foi definido a partir de um orçamento explícito de ~2 horas de CPU para o conjunto dos 8 experimentos (medido empiricamente via testes de fumaça antes da execução), e não por uma limitação artificial de prazo de entrega. Com mais tempo de CPU disponível (ou acesso a GPU), seria esperado que tanto a CNN quanto, especialmente, o ViT melhorassem ainda mais suas métricas, possivelmente estreitando ou invertendo parte das diferenças observadas entre as duas técnicas.

Em resumo, o resultado reforça a literatura: Vision Transformers tendem a precisar de mais dados/épocas (ou de pré-treinamento/transfer learning) para superar CNNs em visão computacional quando treinados do zero, especialmente sob um orçamento de treino limitado como o usado aqui.

7. Vídeo

Link:  video-ia.mp4